

Relatório 8 - OpenMP Sincronização e Sections

1. Enunciado

Implemente um código em OpenMP que realize as seguintes tarefas:

1. Inicialize um vetor que representa uma função qualquer, gerando os valores necessários para o seu processamento.
2. Calcule a integral desse vetor utilizando o método do trapézio.
3. Calcule o vetor que representa a derivada do vetor utilizando o método de diferenças finitas.

As tarefas que não possuem dependência entre si devem ser executadas em paralelo utilizando a diretiva **sections**. Cada tarefa deve utilizar pelo menos duas threads para realizar seu processamento.

Após a conclusão de cada tarefa, o resultado correspondente deve ser impresso por apenas uma thread.

Ao final, discuta as funcionalidades do OpenMP utilizadas na implementação, considerando os seguintes aspectos: - Como foi explorado o paralelismo de tarefas; - Como foi explorado o paralelismo de dados, distribuindo o processamento entre múltiplas threads; - Quais recursos do OpenMP foram utilizados para permitir o aninhamento de regiões paralelas (*nested parallelism*); - Quais diretivas ou mecanismos do OpenMP foram utilizados para garantir que determinadas operações, como a impressão dos resultados, fossem executadas por apenas uma thread; - Quais mecanismos de sincronização estão presentes no código e qual é a função de cada um.

2. Objetivo da Tarefa

O objetivo desta atividade é integrar o **paralelismo de tarefas** (`#pragma omp sections`) com o **paralelismo de dados** (`#pragma omp for`), explorando o **paralelismo aninhado** (*Nested Parallelism*) em OpenMP.

A tarefa exige a execução simultânea de duas operações matemáticas independentes (Cálculo da Integral e Cálculo da Derivada de um vetor de $N = 1.000.000$ posições), onde cada tarefa cria sua própria sub-equipe de threads e utiliza a diretiva `#pragma omp single` para a exibição isolada dos resultados.

3. Metodologia

A solução foi implementada em C com as seguintes etapas:

1. **Inicialização do Vetor:** O vetor `f_x` ($N = 1.000.000$) é gerado representando a função $f(x) = x^2$ no intervalo de $a = 0.0$ a $b = 10.0$, com passo $h = \frac{b-a}{N-1}$. Esta etapa é paralelizada via `#pragma omp parallel for`.
2. **Ativação de Aninhamento:** O paralelismo aninhado é ativado explicitamente via `omp_set_nested(1)`.
3. **Paralelismo de Tarefas (Nível 1):** A diretiva `#pragma omp parallel sections num_threads(2)` cria 2 threads master:
 - **Thread Master 1 (Section A):** Responsável por disparar a tarefa da Integral.
 - **Thread Master 2 (Section B):** Responsável por disparar a tarefa da Derivada.
4. **Paralelismo de Dados (Nível 2 - Sub-equipes):**
 - Dentro de cada `section`, uma nova região paralela `#pragma omp parallel num_threads(2)` divide a carga de trabalho do laço entre uma sub-equipe de 2 threads usando `#pragma omp for`.
 - A tarefa da Integral utiliza a cláusula `reduction(+:soma)`.
5. **Impressão Controlada:** Utiliza-se a diretiva `#pragma omp single` para garantir que apenas uma thread de cada sub-equipe faça a impressão no terminal.

A compilação no macOS seguiu o padrão Clang: `clang -Xpreprocessor -fopenmp -I$(brew --prefix libomp)/include -L$(brew --prefix libomp)/lib -lomp tarefa8.c -o tarefa8`

4. Código-Fonte

```
#include <stdio.h>
#include <stdlib.h>
```

```

#include <omp.h>

#define N 1000000 // Tamanho do vetor (número de subdivisões)

int main() {
    double *f_x = (double *)malloc(N * sizeof(double));
    double *df_x = (double *)malloc(N * sizeof(double));
    double a = 0.0, b = 10.0;
    double h = (b - a) / (N - 1); // Tamanho do passo

    // =====
    // 1. Habilitar o Paralelismo Aninhado
    // =====
    omp_set_nested(1); // Permite que uma thread crie outras threads

    // Inicialização do vetor (Função  $f(x) = x^2$ )
    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        f_x[i] = (a + i * h) * (a + i * h);
    }

    printf("Iniciando processamento de tarefas simultaneas...\n\n");

    // =====
    // PARALELISMO DE TAREFAS (SECTIONS)
    // =====
    // Cria 2 threads no nível mais alto: uma para Integral, outra para Derivada
    #pragma omp parallel sections num_threads(2)
    {
        // -----
        // TAREFA A: CÁLCULO DA INTEGRAL (Trapézio)
        // -----
        #pragma omp section
        {
            double soma = (f_x[0] + f_x[N - 1]) / 2.0;
            double integral = 0.0;

            // Paralelismo de dados: Sub-equipe com pelo menos 2 threads
            #pragma omp parallel num_threads(2)
            {
                #pragma omp for reduction(+:soma)
                for (int i = 1; i < N - 1; i++) {
                    soma += f_x[i];
                }

                // Diretiva para que apenas uma thread da sub-equipe imprima o resultado
                #pragma omp single
                {
                    integral = h * soma;
                    printf("[INTEGRAL] Concluida por 1 thread. Valor = %f\n", integral);
                }
            }
        }
    }
}

```

```

// -----
// TAREFA B: CÁLCULO DA DERIVADA (Diferenças Finitas)
// -----
#pragma omp section
{
    // Paralelismo de dados: Sub-equipe com pelo menos 2 threads
    #pragma omp parallel num_threads(2)
    {
        #pragma omp for
        for (int i = 0; i < N - 1; i++) {
            df_x[i] = (f_x[i + 1] - f_x[i]) / h;
        }

        // Diretiva para que apenas uma thread da sub-equipe imprima o resultado
        #pragma omp single
        {
            df_x[N - 1] = df_x[N - 2]; // Aproximação do último ponto
            printf("[DERIVADA] Concluida por 1 thread. Amostra df_x[N/2] = %f\n", df_x[N/2]);
        }
    }
}

free(f_x);
free(df_x);
return 0;
}

```

5. Resultados e Análises

5.1. Saída de Execução (Terminal)

Ao executar o código compilado, a saída no terminal reflete o paralelismo simultâneo das tarefas e apresenta um aviso informativo da biblioteca OpenMP recente (indicando a depreciação de uma função legada):

```
OMP: Info #276: omp_set_nested routine deprecated, please use omp_set_max_active_levels instead. Iniciando processamento de tarefas simultaneas...
```

```
[DERIVADA] Concluida por 1 thread. Amostra df_x[N/2] = 10.000020 [INTEGRAL] Concluida por 1 thread. Valor = 333.333333
```

(A integral de x^2 de 0 a 10 converge para 333,33, e a aproximação por diferenças finitas para a derivada em $x = 5$ resulta em 10.000020 devido à precisão de ponto flutuante do passo h).

5.2. Discussão das Funcionalidades OpenMP

Conforme exigido pelo enunciado, a arquitetura do programa explora cinco dimensões fundamentais do OpenMP:

1. Exploração do Paralelismo de Tarefas O paralelismo de tarefas foi modelado com a diretiva `#pragma omp parallel sections`. Essa abordagem permite que fluxos de instruções completamente independentes (calcular uma integral vs. calcular uma derivada) sejam despachados para diferentes núcleos da CPU. A thread master da aplicação cria uma equipe de 2 threads, onde cada uma recebe um bloco `#pragma omp section` exclusivo para processar simultaneamente.

2. Exploração do Paralelismo de Dados Dentro de cada “tarefa macro” (Integral e Derivada), há o processamento massivo de um vetor de 1.000.000 de posições. O paralelismo de dados foi aplicado utilizando `#pragma omp parallel num_threads(2)` seguido de `#pragma omp for`. Isso faz com que as duas threads da

sub-equipe dividam fatias iguais do laço de iteração (500.000 índices para cada), acelerando o preenchimento do vetor e o acúmulo da área do trapézio.

3. Aninhamento de Regiões Paralelas (*Nested Parallelism*) Por padrão, o OpenMP desativa a criação de novas equipes de threads se o código já estiver dentro de uma região paralela. Para permitir que as threads que cuidavam das *sections* criassem suas próprias sub-equipes de 2 threads para o laço *for*, foi necessário invocar a função da API `omp_set_nested(1)` no início do *main*. Estruturalmente, o aninhamento ocorreu ao colocar um `#pragma omp parallel` isolado no interior do escopo de um `#pragma omp section`.

4. Diretivas de Execução Única (Impressão) Para garantir que o `printf` final de cada cálculo fosse exibido apenas uma vez (e não duplicado pelas 2 threads da sub-equipe atuando no paralelismo de dados), utilizou-se a diretiva `#pragma omp single`. Essa diretiva garante que o bloco de código interno seja executado pela primeira thread que o alcançar, enquanto as demais ignoram o bloco.

5. Mecanismos de Sincronização Presentes * **reduction(+:soma):** Mecanismo de sincronização de dados que cria cópias locais da variável *soma* e realiza uma fusão atômica global ao final do laço da integral, prevenindo condições de corrida na agregação do resultado. * **Barreira Implícita do single:** Ao final da região `#pragma omp single`, há uma sincronização automática (barreira implícita). A thread que realizou a impressão e a thread que a ignorou se encontram no final do bloco antes de prosseguirem, garantindo consistência no fluxo. * **Barreira Implícita do sections e do for:** Da mesma forma, existem barreiras implícitas ao fim dos blocos *sections* e *for*, assegurando que a memória principal só seja finalizada quando todas as threads concluírem suas fatias de processamento.

6. Conclusão

A atividade demonstra a robustez e a flexibilidade do modelo de concorrência do OpenMP ao combinar paralelismo de tarefas (fluxos de controle distintos) e paralelismo de dados (divisão de laços extensos). A utilização do paralelismo aninhado (*nested parallelism*) provou ser uma estratégia poderosa para maximizar a ocupação dos núcleos da CPU: mesmo havendo apenas duas tarefas de alto nível, o sistema alocou um total de quatro *worker threads* (duas sub-threads para cada tarefa) para acelerar as operações nos vetores. O controle preciso através das cláusulas *reduction* e *single* evitou o comprometimento dos dados e a poluição do console, garantindo que algoritmos matemáticos complexos sejam paralelizados com total correteza.